

TP : Gestion des processus - Système d'exploitation 2

Réponses aux exercices

Exercice 1

1. Fonction principale du programme et rôle de `fork()` et `sleep()`

La fonction principale du programme est de créer un processus fils avec `fork()` et de faire dormir les processus père et fils pour des durées différentes spécifiées en arguments.

- `fork()` : Duplique le processus courant, créant un processus fils identique au père. Retourne 0 dans le fils, le PID du fils dans le père, ou -1 en cas d'erreur.
- `sleep()` : Suspend l'exécution du processus appelant pour un nombre spécifié de secondes.

2. Avec arguments 10 20

- **Avant la fin du père** (dans les 10 premières secondes) :
 - Père : PPID = PID du shell parent
 - Fils : PPID = PID du père
- **Avant la fin du fils** (entre 10 et 20 secondes) :
 - Le père est terminé
 - Le fils devient "orphelin" et est adopté par `init` (PPID = 1)

3. Avec arguments 10 0

Le fils dort 0 seconde, donc s'exécute immédiatement et termine. Avant la fin du père (dans les 10 premières secondes), `ps -la` montre le fils comme processus **zombie** (état Z) car le père n'a pas encore appelé `wait()` pour récupérer son statut.

4. Situation attente `_pere=5`, attente `_fils=10`

- Père attend 5 secondes
- Fils attend 10 secondes
- **Ordre d'affichage** :
 1. "pere attente finie" (après 5 secondes)
 2. "fils attente finie" (après 10 secondes)

5. Modification pour afficher PID, propriétaire et groupe

```
1 #include <sys/types.h>
2 #include <unistd.h>
3 #include <stdio.h>
4
5 // Dans le case 0: (fils)
```

```
6 printf("Fils - PID: %d, PPID: %d, UID: %d, GID: %d\n",
7       getpid(), getppid(), getuid(), getgid());
8
9 // Dans le default: (p re)
10 printf("P re - PID: %d, PPID: %d, UID: %d, GID: %d\n",
11        getpid(), getppid(), getuid(), getgid());
```

Effet sur la sortie : Les deux processus afficheront les mêmes UID et GID (hérités du parent) mais des PID différents.

Exercice 2

1. Commande pour afficher l'arbre des processus

```
1 # Affiche avec hi rarchie
2 ps -ejH
3
4 # Affiche en format arbre
5 ps axjf
6
7 # Commande d di e
8 pstree
```

2. Programme pour créer une arborescence

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <stdlib.h>
4
5 int main() {
6     pid_t pid1, pid2;
7
8     // Processus A
9     printf("A - PID: %d, PPID: %d\n", getpid(), getppid());
10
11    // Cr ation du processus B
12    pid1 = fork();
13    if (pid1 == 0) { // Processus B
14        printf("B - PID: %d, PPID: %d\n", getpid(), getppid());
15
16        // Cr ation du processus C
17        pid2 = fork();
18        if (pid2 == 0) { // Processus C
19            printf("C - PID: %d, PPID: %d\n", getpid(), getppid());
20        }
21        exit(0);
22    }
23
24    // Attente du fils B
25    wait(NULL);
26    return 0;
27 }
```

3. Programme qui affiche l'arbre du processus courant

```
1 #include <stdlib.h>
2 #include <stdio.h>
3
4 int main() {
5     printf("Arbre des processus pour PID %d:\n", getpid());
6     system("pstree -p $$"); // $$ = PID du processus courant
```

```
7     return 0;  
8 }
```

Exercice 3

1. & 2. Programme avec création de processus et attente

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/wait.h>
4 #include <stdlib.h>
5
6 int main() {
7     pid_t pid = fork();
8
9     if (pid == 0) { // Processus P2 (fils)
10         printf("Fils P2 - PID: %d\n", getpid());
11         exit(0); // Terminaison correcte
12     }
13     else { // Processus P1 (p re)
14         int status;
15         wait(&status);
16
17         if (WIFEXITED(status)) {
18             printf("P re - Fils termin avec code: %d\n",
19                   WEXITSTATUS(status));
20         }
21     }
22     return 0;
23 }
```

3. Fils avec code retour 3

Modifier la ligne dans le fils :

```
1 exit(3); // Au lieu de exit(0)
```

Le père affichera : "Père - Fils terminé avec code : 3"

4. kill -15 pid_fil

```
1 // Dans le fils :
2 sleep(10);
3 // Le fils sera tu par kill -15 avant de terminer
4
5 // Dans le p re apr s wait() :
6 if (WIFSIGNALED(status)) {
7     printf("Fils termin par signal: %d\n",
8           WTERMSIG(status)); // Affichera 15 (SIGTERM)
9 }
```

5. kill -9 pid_fil

Même code, mais le signal sera 9 (SIGKILL). Le père détectera la terminaison par signal 9.

Exercice 4

1. Processus zombie

Un processus zombie est un processus qui a terminé son exécution mais dont le parent n'a pas encore appelé `wait()` pour récupérer son statut de terminaison. Il reste dans la table des processus avec l'état "Z" (zombie) jusqu'à ce que le parent récupère son statut.

2. Programme créant un zombie

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <stdlib.h>
4
5 int main() {
6     pid_t pid = fork();
7
8     if (pid == 0) { // Fils
9         printf("Fils termin , devenant zombie\n");
10        exit(0);
11    }
12    else { // P re
13        printf("P re - PID: %d, Fils PID: %d\n",
14               getpid(), pid);
15        printf("Le fils est maintenant zombie "
16               "(pas de wait())\n");
17        sleep(30); // Temps pour observer avec ps
18    }
19    return 0;
20 }
```

3. Élimination du zombie

```
1 // Ajouter dans le p re , apr s le printf :
2 int status;
3 wait(&status); // R cup re le statut du fils
4 // ou
5 waitpid(pid, &status, 0);
```

Exercice 5

1. Exécution du programme

Le programme original :

1. Affiche un message de début avec son PID
2. Formate une chaîne avec son PID
3. Appelle `execl()` pour exécuter `/bin/echo`
4. Si `execl()` réussit, les `printf()` suivants ne sont PAS exécutés (l'image du processus est remplacée)
5. Si `execl()` échoue, les messages d'échec sont affichés

Sortie typique :

```
[1234] - Debut du test exec()
Execution d'un test exec pid=[1234]
```

2. Avec `"/bin/ech"` (commande inexistante)

`execl()` échoue car la commande n'existe pas. Tous les `printf()` sont exécutés :

Sortie :

```
[1234] - Debut du test exec()
[1234] - Echec de la fonction execl()
[1234] - Fin du test exec()
```

Exercice 6

1. Avec fork() et execl()

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/wait.h>
4 #include <stdlib.h>
5
6 int main() {
7     pid_t pid = fork();
8
9     if (pid == 0) { // Fils
10         // Ex cute ps -l
11         execl("/bin/ps", "ps", "-l", NULL);
12         perror("execl chou ");
13         exit(1);
14     }
15     else { // P re
16         int status;
17         wait(&status);
18
19         if (WIFEXITED(status)) {
20             printf("Commande ps termin e avec code: %d\n",
21                    WEXITSTATUS(status));
22         }
23     }
24     return 0;
25 }
```

2. Avec execv()

```
1 // Dans le fils :
2 char *args[] = {"ps", "-l", NULL};
3 execv("/bin/ps", args);
```


Exercice 7

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/wait.h>
4 #include <stdlib.h>
5
6 int main() {
7     pid_t pid = fork();
8
9     if (pid == 0) { // Fils
10         // Remplace l'image par ls -l /
11         execl("/bin/ls", "ls", "-l", "/", NULL);
12         perror("execl chou ");
13         exit(1);
14     }
15     else { // P re
16         wait(NULL); // Attend la fin du fils
17         printf("Commande ls termin e\n");
18     }
19     return 0;
20 }
```

Ce programme remplace l'image du processus fils par `ls -l /`, affichant le contenu détaillé du répertoire racine. Le père attend la fin de l'exécution du fils avant de terminer.